

# Docker Commands, Tags & Flags

## COMMANDS:

1. **Pull:** downloads an image from a registry (like Docker Hub) to your local machine.

```
docker pull <image_name>
```

2. **Images:** list all images stored locally on your machine

```
docker images
```

3. **Run:** create and start a new container from an image

```
docker run <image_name>
```

4. **Ps (process status):** list all running containers

```
docker ps (Add -a flag to see the stopped ones also)
```

5. **Start:** start an existing (stopped) container

```
docker start CONTAINER_NAME/CONTAINER_ID
```

6. **Stop:** stop a running container (stops gracefully)

```
docker stop CONTAINER_NAME/CONTAINER_ID
```

7. **Kill:** Stops a running container. (immediately)

```
docker kill CONTAINER_NAME/CONTAINER_ID
```

8. **Restart:** Restarts a container

```
docker restart CONTAINER_NAME/CONTAINER_ID
```

## 9. Stop all containers

```
docker stop $(docker ps -q)
docker ps -q // Return ids of all running containers
```

## 10. Rm: Remove (delete) one or more containers

```
docker rm CONTAINER_NAME/CONTAINER_ID
```

## 11. Rmi: Remove (delete) one or more images from your local machine

```
docker rmi IMAGE_NAME/IMAGE_ID
```

**NOTE:** Before deleting an image, we have to delete the container first if any were created with that image. Add **-f** to force remove (delete both container and image)

## 12. Prune: deletes unused Docker objects to reclaim disk space.

### Types:

- **Image:** removes dangling/unused images
- **Container:** removes stopped containers
- **Volume:** deletes unused volumes (Dangerous, deletes your data)
- **Network:** removes unused networks
- **System:** Removes stopped containers, unused networks, dangling images, and build cache

```
docker system prune
docker image prune
docker container prune
docker volume prune
docker network prune

# KEY FLAGS
-a / --all # removes ALL unused images, not just dangling ones
-f / --force # skips confirmation prompt
```

**Note:** There are many more commands. Use the `docker` command, it will list all the available commands, and if to learn more about a specific command, then **`docker <command> --help`**

## Troubleshooting Commands.

**13. Logs:** show the logs (output) from a container

```
docker logs CONTAINER_NAME/CONTAINER_ID
```

**NOTE:** Add -f (follow). Instead of just showing old logs once and stopping, it will **keep streaming new logs in real-time** as they happen.

**14. Exec:** run a command inside an already running container

```
docker exec -it CONTAINER_ID/NAME bash # Access Bash
docker exec -it CONTAINER_ID/NAME sh # Access Shell
```

## IMAGE TAGS / VERSIONS

In Docker, **tags** are like version labels for images.

- **Format:** image:tag
- If you don't give a tag, Docker assumes latest.

```
docker pull ubuntu:20.04 # Ubuntu 20.04
docker pull ubuntu # same as ubuntu:latest
```

**So:** Image = name, Tag = version.

## FLAGS

Flags are **extra switches you add to a command to change how it behaves**. Think of them like “**settings**” or “**options**” you turn on/off when running a command. Flags are used with a – (minus) sign

**For docker run command**

- **-d:** run in background (detached mode)

```
docker run -d nginx
```

- **-it:** interactive terminal

```
docker run -it ubuntu bash
```

- **-p:** map ports (host:container)

```
docker run -p 8080:80 nginx
```

- **-v:** mount volume (host:path)

```
docker run -v $(pwd):/app node
```

- **-e:** set environment variable

```
docker run -e NODE_ENV=production node
```

- **--env-file:** provide an env file instead of writing one by one.

```
docker run --env-file <env_file_path>
```

- **--rm:** automatically deletes the container when it stops.

```
docker run --rm ubuntu echo "hello."  
# Container runs, prints "hello.", exits, and is immediately removed.
```

- **--name:** give the container a custom name

```
docker run --name myapp nginx
```

### For docker ps

- **-a:** show all containers (even stopped ones)

```
docker ps -a
```

### For docker rm / rmi

- **-f:** force remove container/image

```
docker rm -f mycontainer  
docker rmi -f myimage
```

